

Assignment: 4 Way Intersection Control Raspberry Pi 5

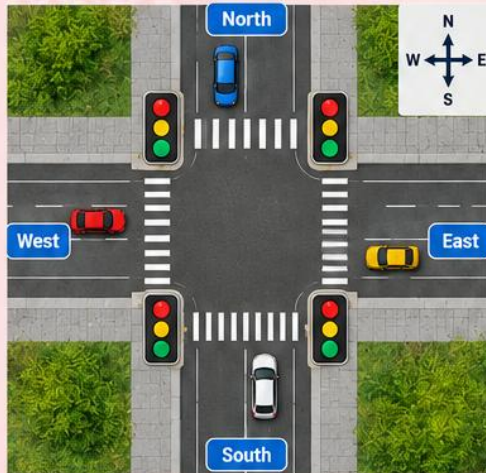
1. Project Overview

In this assignment, you will create a 4-way traffic intersection similar to a real intersection in Montreal. Each direction (North, East, South, West) will have a set of traffic lights (Red, Yellow, Green LEDs) controlled by a Raspberry Pi 5. The lights will automatically switch every 10 seconds in a safe and correct sequence.

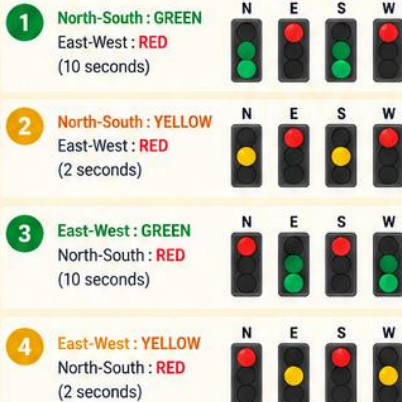
2. Real-World Intersection Example (Montreal)



3. Intersection Layout



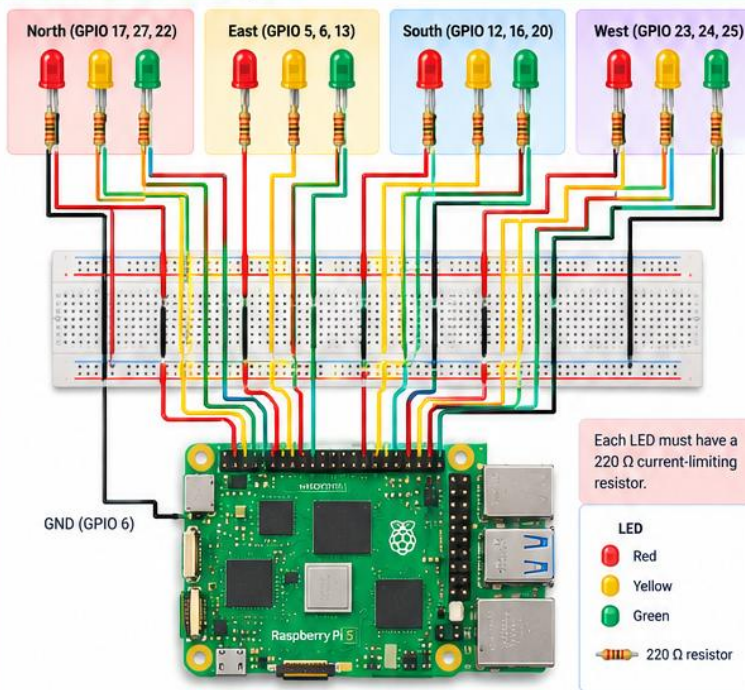
4. Traffic Light Sequence



5. Requirements

- 1 Use Raspberry Pi 5 to control 12 LEDs (3 LEDs per direction × 4 directions).
- 2 Automatically switch the traffic lights every 10 seconds (with 2-second yellow transition).
- 3 Implement proper traffic light sequence (Green → Yellow → Red).
- 4 Use real GPIO pins with proper resistors.
- 5 Find and handle all edge cases.
- 6 Ensure no conflicting green lights.
- 7 Consider power-on startup state (all red).
- 8 Use clear, modular, and well-commented Python code.
- 9 (Optional) Add a pedestrian button and crossing signal.
- 10 Create a short report explaining your design and testing.

6. Hardware Connection Layout



7. GPIO Pin Assignment

Direction	Red LED	Yellow LED	Green LED
North	GPIO 17	GPIO 27	GPIO 22
East	GPIO 5	GPIO 6	GPIO 13
South	GPIO 12	GPIO 16	GPIO 20
West	GPIO 23	GPIO 24	GPIO 25

Common GND: Use GPIO 6 (or any GND pin) for all LED cathodes.

9. Edge Cases to Consider

- 1 No two opposite directions should have green at the same time.
- 2 Proper yellow transition between green and red.
- 3 Initial state on power-up (all red).
- 4 What happens if the program stops unexpectedly?
- 5 Handle GPIO errors or incorrect pin setup.
- 6 Consider pedestrian crossing (optional).
- 7 Ensure proper timing even if system is busy.
- 8 Test the full cycle multiple times for reliability.

10. Deliverables

- ☐ Working hardware setup (Raspberry Pi 5 + LEDs)
- ☐ Python program
- ☐ Circuit diagram (photo or drawing)
- ☐ Short report (logic, edge cases, testing results)
- ☐ (Optional) Video demonstration

11. Evaluation Criteria

Functionality (correct sequence)	40%
Handling of edge cases	20%
Code quality (structure, comments)	20%
Circuit neatness	10%
Report and demonstration	10%

1. Assignment Scenario

You have been asked to prototype the traffic-light controller for a four-way urban intersection in Montreal. The prototype will use a Raspberry Pi 5 and twelve LEDs: one red, one yellow, and one green LED for each of the North, South, East, and West approaches. Your program must operate continuously, switch traffic right-of-way between the North-South and East-West directions, and always maintain a safe transition sequence.

The main green phase for each direction group is 10 seconds. A short yellow phase must occur before that group changes to red. Students must determine and justify a safe transition strategy, including whether a short all-red clearance interval is needed.

2. Hardware Requirements

Component	Quantity	Purpose
Raspberry Pi 5	1	Traffic controller
Red LEDs	4	Stop indication
Yellow LEDs	4	Transition/caution indication
Green LEDs	4	Go indication
220–330 Ω resistors	12	One current-limiting resistor per LED
Breadboard(s)	1–2	Circuit construction
Jumper wires	As needed	GPIO and ground connections
Raspberry Pi power supply	1	Power

3. Intersection Model

Treat North and South as one non-conflicting traffic group and East and West as the other. North and South may be green together. East and West may be green together. North-South and East-West must never be green at the same time.

State	North	South	East	West
1. NS GO (10 s)	GREEN	GREEN	RED	RED
2. NS TRANSITION	YELLOW	YELLOW	RED	RED
3. Clearance (student decision)	RED	RED	RED	RED
4. EW GO (10 s)	RED	RED	GREEN	GREEN
5. EW TRANSITION	RED	RED	YELLOW	YELLOW
6. Clearance (student decision)	RED	RED	RED	RED

4. Hardware Connection Layout

Use the following illustrated layout as a visual guide. The GPIO assignment table below is the required reference for your implementation. Each LED must be connected through its own 220–330 Ω resistor, and all LED cathodes must return to a Raspberry Pi ground pin.

5. Suggested GPIO Assignment

Direction	Red LED	Yellow LED	Green LED
North	GPIO 17	GPIO 27	GPIO 22
South	GPIO 12	GPIO 16	GPIO 20
East	GPIO 5	GPIO 6	GPIO 13
West	GPIO 23	GPIO 24	GPIO 25

Ground: use any suitable Raspberry Pi GND pin. Use BCM numbering consistently in your Python program.

6. Required Traffic-Light Behaviour

- At startup, initialize every traffic LED to a known safe state. The recommended initial state is ALL RED.
- North-South green phase: North and South green; East and West red. Maintain this green phase for 10 seconds.
- North-South yellow phase: North and South change from green to yellow while East and West remain red.
- Before granting green to East-West, ensure North and South are red.
- East-West green phase: East and West green; North and South red. Maintain this green phase for 10 seconds.
- East-West yellow phase: East and West change from green to yellow while North and South remain red.
- Return to a safe all-red state before beginning the next North-South cycle.
- The program must repeat continuously until the operator stops it.

7. Important Interpretation of the 10-Second Requirement

For this assignment, the 10-second requirement applies to the GREEN right-of-way phase. Yellow and optional all-red clearance intervals are additional transition time. Students must choose reasonable transition durations and document their choices. The controller must never jump directly from GREEN in one direction to GREEN in the conflicting direction.

8. Edge Cases You Must Find, Test, and Explain

Edge Case	Question to Address
Power-on/startup	What should every light show before the first normal phase begins?
Conflicting greens	How will your code guarantee that North-South and East-West can never be green together?
Green → Yellow → Red	Can any phase skip yellow or change directly from green to a conflicting green?
All-red clearance	Is a brief all-red interval useful between conflicting movements? Justify your design.
Program interruption	What happens if Ctrl+C stops the program during a green phase?
Unexpected exception	How will your program place the intersection into a safe state before GPIO cleanup?
Invalid/incorrect pin setup	How will you detect or troubleshoot a wrong GPIO assignment or loose wire?
LED failure	How would the system behave if one LED fails? What would a real safety-critical controller need?
Timing drift/blocking	Does any unnecessary code delay the state transitions?
Restart after failure	Does the controller restart from a known safe state or from an unknown state?

9. Program Design Requirements

- Use functions to avoid repeating GPIO commands.
- Represent the four directions and their LED pins using dictionaries, lists, tuples, or another clear data structure.
- Create a function that can set one direction to RED, YELLOW, or GREEN.
- Create a safe-state function that turns every approach RED.
- Use a loop to repeat the complete intersection cycle.
- Use try/except/finally so the GPIO pins are cleaned up when the program exits.
- Print the current state to the terminal so the instructor can follow the sequence during testing.
- Add meaningful comments and use descriptive variable/function names.

10. Introductory Raspberry Pi 5 Python Code

The following code is only a starting structure. It initializes the pins and provides a reusable light-setting function. You must complete the state sequence, timing, safety checks, exception handling, and full-cycle logic.

```
from gpiozero import LED
from time import sleep

# =====
# 1. GPIO PIN CONFIGURATION
# =====

# Each traffic direction has three LEDs:
# Red, Yellow, and Green.
#
# gpiozero uses BCM GPIO numbering directly.

LIGHTS = {
    "north": {
        "red": LED(17),
        "yellow": LED(27),
        "green": LED(22)
    },

    "south": {
        "red": LED(12),
        "yellow": LED(16),
        "green": LED(20)
    },

    "east": {
        "red": LED(5),
        "yellow": LED(6),
        "green": LED(13)
    },

    "west": {
        "red": LED(23),
        "yellow": LED(24),
```

```

        "green": LED(25)
    }
}

# =====
# 2. TIMING CONFIGURATION
# =====

# Main traffic timing values.
#
# Each traffic direction gets a total traffic window
# of approximately 10 seconds:
#
# 8 seconds GREEN
# 2 seconds YELLOW

GREEN_TIME = 8
YELLOW_TIME = 2

# Optional safety clearance interval.
#
# During this period, ALL traffic directions are RED.
#
# You may set this to:
# 0 -> no extra clearance
# 1 -> 1 second all-red safety period

ALL_RED_TIME = 1

# =====
# 3. FUNCTION: SET ONE DIRECTION TO ONE COLOR
# =====

def set_light(direction_name, colour):
    """
    Turn ON only the requested colour for one traffic direction.

    Example:
        set_light("north", "green")

    Result:
        North Green -> ON
        North Yellow -> OFF
        North Red -> OFF
    """

    direction = LIGHTS[direction_name]

    for colour_name, led in direction.items():

```

```

        if colour_name == colour:
            led.on()
        else:
            led.off()

# =====
# 4. FUNCTION: ALL RED
# =====

def all_red():
    """
    Put the entire intersection into a safe state.

    North -> RED
    South -> RED
    East  -> RED
    West  -> RED
    """

    set_light("north", "red")
    set_light("south", "red")
    set_light("east", "red")
    set_light("west", "red")

    print("\n=====")
    print("SAFE STATE: ALL DIRECTIONS RED")
    print("=====")

# =====
# 5. FUNCTION: NORTH-SOUTH GREEN
# =====

def north_south_green():
    """
    Allow North-South traffic to move.

    North -> GREEN
    South -> GREEN
    East  -> RED
    West  -> RED
    """

    set_light("north", "green")
    set_light("south", "green")

    set_light("east", "red")
    set_light("west", "red")

    print("\n-----")
    print("NORTH-SOUTH : GREEN")
    print("EAST-WEST   : RED")

```

```

print("-----")

# =====
# 6. FUNCTION: NORTH-SOUTH YELLOW
# =====

def north_south_yellow():
    """
    Warn North-South traffic that the green phase is ending.

    North -> YELLOW
    South -> YELLOW
    East   -> RED
    West   -> RED
    """

    set_light("north", "yellow")
    set_light("south", "yellow")

    set_light("east", "red")
    set_light("west", "red")

    print("\n-----")
    print("NORTH-SOUTH : YELLOW")
    print("EAST-WEST    : RED")
    print("-----")

# =====
# 7. FUNCTION: EAST-WEST GREEN
# =====

def east_west_green():
    """
    Allow East-West traffic to move.

    North -> RED
    South -> RED
    East   -> GREEN
    West   -> GREEN
    """

    set_light("north", "red")
    set_light("south", "red")

    set_light("east", "green")
    set_light("west", "green")

    print("\n-----")
    print("NORTH-SOUTH : RED")
    print("EAST-WEST    : GREEN")

```

```

print("-----")

# =====
# 8. FUNCTION: EAST-WEST YELLOW
# =====

def east_west_yellow():
    """
    Warn East-West traffic that the green phase is ending.

    North -> RED
    South -> RED
    East   -> YELLOW
    West   -> YELLOW
    """

    set_light("north", "red")
    set_light("south", "red")

    set_light("east", "yellow")
    set_light("west", "yellow")

    print("\n-----")
    print("NORTH-SOUTH : RED")
    print("EAST-WEST    : YELLOW")
    print("-----")

# =====
# 9. FUNCTION: OPTIONAL SAFETY CLEARANCE
# =====

def clearance():
    """
    Place all traffic lights on RED before changing
    the right-of-way to the other direction.

    This helps prevent conflicting traffic movement.
    """

    all_red()

    if ALL_RED_TIME > 0:
        print(f"Safety clearance: {ALL_RED_TIME} second(s)")
        sleep(ALL_RED_TIME)

# =====
# 10. FUNCTION: RUN NORTH-SOUTH PHASE
# =====

```

```

def run_north_south_phase():
    """
    Complete North-South traffic phase:

    GREEN  -> 8 seconds
    YELLOW -> 2 seconds
    RED     -> clearance
    """

    north_south_green()

    print(f"North-South green for {GREEN_TIME} seconds")
    sleep(GREEN_TIME)

    north_south_yellow()

    print(f"North-South yellow for {YELLOW_TIME} seconds")
    sleep(YELLOW_TIME)

    clearance()


# =====
# 11. FUNCTION: RUN EAST-WEST PHASE
# =====

def run_east_west_phase():
    """
    Complete East-West traffic phase:

    GREEN  -> 8 seconds
    YELLOW -> 2 seconds
    RED     -> clearance
    """

    east_west_green()

    print(f"East-West green for {GREEN_TIME} seconds")
    sleep(GREEN_TIME)

    east_west_yellow()

    print(f"East-West yellow for {YELLOW_TIME} seconds")
    sleep(YELLOW_TIME)

    clearance()


# =====
# 12. FUNCTION: SAFE SHUTDOWN
# =====

```

```

def safe_shutdown():
    """
    Attempt to leave the intersection in a safe state
    before the Python program exits.
    """

    print("\nTraffic controller is shutting down.")

    all_red()

    print("Intersection placed in safe ALL-RED state.")

# =====
# 13. MAIN PROGRAM
# =====

try:

    print("\n=====")
    print("4-WAY TRAFFIC INTERSECTION CONTROLLER")
    print("Raspberry Pi 5 + gpiozero")
    print("=====")

    # -----
    # SAFE STARTUP
    # -----

    # Never immediately start with a GREEN signal.
    # Begin with all directions RED.

    all_red()

    print("System starting...")
    sleep(2)

    # -----
    # CONTINUOUS TRAFFIC CONTROL LOOP
    # -----

    cycle_number = 1

    while True:

        print("\n")
        print("=====")
        print(f"TRAFFIC CYCLE {cycle_number}")
        print("=====")

        # -----
        # NORTH-SOUTH TRAFFIC

```

```

# -----

run_north_south_phase()

# -----
# EAST-WEST TRAFFIC
# -----

run_east_west_phase()

# Increase cycle counter

cycle_number += 1

# =====
# 14. KEYBOARD INTERRUPT
# =====

except KeyboardInterrupt:

    print("\nCTRL+C detected.")

# =====
# 15. OTHER UNEXPECTED ERRORS
# =====

except Exception as error:

    print("\nUnexpected error occurred:")
    print(error)

# =====
# 16. FINAL CLEANUP / SAFE STATE
# =====

finally:

    safe_shutdown()

    print("Traffic controller stopped.")

```

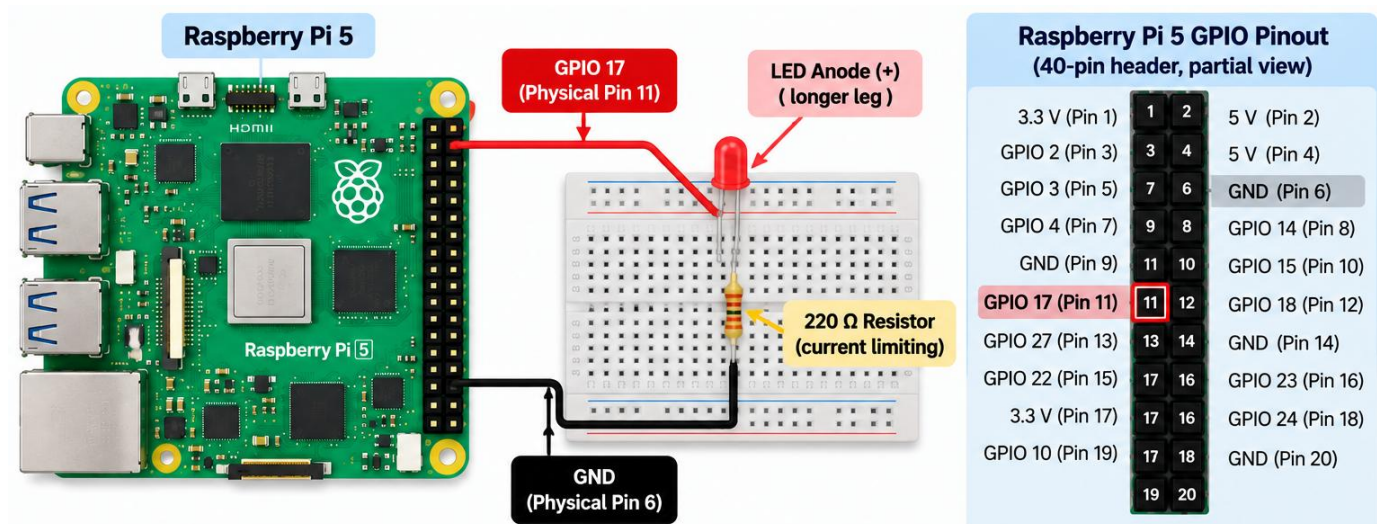
11. Required State-Machine Planning

Before coding, draw a state diagram or flowchart. Your design should contain at least these logical states:

- START / ALL RED
- NORTH-SOUTH GREEN
- NORTH-SOUTH YELLOW
- ALL RED / CLEARANCE
- EAST-WEST GREEN
- EAST-WEST YELLOW
- ALL RED / CLEARANCE
- REPEAT

For every state, identify: (1) which LEDs are ON, (2) which LEDs are OFF, (3) how long the state lasts, and (4) the only legal next state.

12. Single LED Connection Diagram



13. Testing Requirements

- Test each individual LED before running the complete intersection program.
- Verify all four RED LEDs can be activated simultaneously.
- Verify the North-South group independently.
- Verify the East-West group independently.
- Run at least three complete automatic cycles.
- Record evidence that conflicting green LEDs never occur.
- Stop the program during a green phase and verify your shutdown/fail-safe behaviour.
- Restart the program and verify that it begins from a safe known state.
- Document at least five edge-case tests with expected and observed results.

14. Required Evidence

Include the following in your short report:

- A clear Video of your Raspberry Pi 5 and breadboard circuit and working prototype.

- A circuit/wiring diagram with GPIO numbers.
- A state diagram or flowchart.
- A table describing your chosen timing values.
- Screenshots or photos from testing.

15. Deliverables

- traffic_intersection.py
- Circuit diagram (image/PDF or clearly labelled drawing)
- State-machine/flowchart diagram
- Short report (design, timing, edge cases, tests, results)
- Photo of the working Raspberry Pi 5 prototype
- In-class demonstration of at least two complete cycles

16. Submission Structure

```
YOURNAME_GroupName_IDs/
├── traffic_intersection.py
├── circuit_diagram.png
├── state_diagram.png
├── report.pdf
├── Video/
│   └── prototype.mp4
```

Submit as: GroupName_StudentIDs.zip

17. Evaluation Rubric

Criterion	Marks
Correct GPIO circuit and 12 LED operation	15
Correct North-South / East-West sequencing	20
10-second green phases and proper yellow transitions	10
Safety: no conflicting green states	15
Edge-case identification and handling	15
Testing evidence and troubleshooting	5
Circuit/state diagrams and short report	10
Working demonstration	10
TOTAL	100

18. Safety and Good Practice

This is a classroom prototype, not a controller for a real roadway. Never connect the Raspberry Pi directly to mains-powered traffic equipment. Use only low-voltage LEDs and instructor-approved components. GPIO pins are 3.3 V logic and must not be exposed to inappropriate voltages. Always use a current-limiting resistor with each LED.

Build the prototype → prove the safety logic → test the edge cases → demonstrate reliable operation.

19. Instructions

1. Write your **team name and student IDs as comments at the beginning of every Python file.**

2. Your application must execute without errors.
3. Properly comment important sections of your Python code.
4. Submit the complete project as one ZIP file before the deadline.
5. You need to check off the assignment with your teacher in person within 1 week after the due date. Failing to do this will result in an automatic zero / 0.